AIAC Assignment 13.3

Name: M.JAYANTH

HT No: 2303A52485

Batch: 32

Task Description #1 (Refactoring – Removing Code Duplication)
• Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.
• Instructions:
o Prompt AI to identify duplicate logic and replace it with functions or classes.
o Ensure the refactored code maintains the same output.
o Add docstrings to all functions.
• Sample Legacy Code:

```
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

• Expected Output:
o Refactored code with a reusable function and no duplication.
o Well documented code

```
print("Area of Rectangle:", 5 * 10)

print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))

# Refactor the above code and add docstrings to the function

def calculate_rectangle_properties(length, width):
    """
    Calculate the area and perimeter of a rectangle.
    Parameters:
    length (float): The length of the rectangle.
    width (float): The width of the rectangle.
    Returns:
    tuple: A tuple containing the area and perimeter of the rectangle.
    """
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter
rectangles = [(5, 10), (7, 12), (10, 15)]
for length, width in rectangles:
    area, perimeter = calculate_rectangle_properties(length, width)
    print(f"Area of Rectangle: {area}")
    print(f"Perimeter of Rectangle: {perimeter}")
```

"---1.py done---"

Area of Rectangle: 50

Perimeter of Rectangle: 30

Area of Rectangle: 84

Perimeter of Rectangle: 38

Area of Rectangle: 150

Perimeter of Rectangle: 50

Area of Rectangle: 50

Perimeter of Rectangle: 30

Area of Rectangle: 84

Perimeter of Rectangle: 38

Area of Rectangle: 150

Perimeter of Rectangle: 50

---1.py done---

Task Description #2 (Refactoring – Extracting Reusable Functions)
• Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.
• Instructions:
o Identify repeated or related logic and extract it into reusable functions.
o Ensure the refactored code is modular, easy to read, and documented with docstrings.
• Sample Legacy Code:

# Legacy script with inline repeated logic
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)
price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
• Expected Output:
o Code with a function calculate_total(price) that can be reused for multiple price inputs.
o Well documented code

```
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)
price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)

# Refactor the above code and add docstrings to the function

def calculate_total_price(price):
    """
    Calculate the total price including tax.
    Parameters:
    price (float): The original price of the item.
    Returns:
    float: The total price including tax.
```

```
    """
    tax = price * 0.18
    total = price + tax
    return total
prices = [250, 500]
for price in prices:
    total_price = calculate_total_price(price)
    print(f"Total Price: {total_price}")
```

Total Price: 295.0

Total Price: 590.0

Total Price: 295.0

Total Price: 590.0

---2.py done---

Task Description #3: Refactoring Using Classes and Methods (Eliminating Redundant Conditional Logic)

Refactor a Python script that contains repeated if–elif–else grading logic by implementing a structured, object-oriented solution using a class and a method.

Problem Statement

The given script contains duplicated conditional statements used to assign grades based on student marks. This redundancy violates clean code principles and reduces maintainability.

You are required to refactor the script using a class-based design to improve modularity, reusability, and readability while preserving the original grading logic.

Mandatory Implementation Requirements

1. Class Name: GradeCalculator
2. Method Name: calculate_grade(self, marks)
3. The method must:
o Accept marks as a parameter.

o Return the corresponding grade as a string.
o The grading logic must strictly follow the conditions below:
▪ Marks ≥ 90 and ≤ 100 → "Grade A"
▪ Marks ≥ 80 → "Grade B"
▪ Marks ≥ 70 → "Grade C"
▪ Marks ≥ 40 → "Grade D"
▪ Marks ≥ 0 → "Fail"

Note: Assume marks are within the valid range of 0 to 100.

4. Include proper docstrings for:
o The class
o The method (with parameter and return descriptions)
5. The method must be reusable and called multiple times without rewriting conditional logic.

• Given code:

```
marks = 85
if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
marks = 72
if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
```

Expected Output:

- Define a class named GradeCalculator.
- Implement a method calculate_grade(self, marks) inside the class.

- Create an object of the class.
- Call the method for different student marks.
- Print the returned grade values.

```
marks = 85
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
marks = 72
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
"""
Refactor the grading logic using a class named GradeCalculator with a reusable
method
calculate_grade(self, marks) that returns the correct grade based on the given
marks.
Add proper docstrings for the class and method.
"""
class GradeCalculator:
    """A class to calculate the grade based on marks.
    """
    def calculate_grade(self, marks):
        """
        Calculate the grade based on the given marks.
        Parameters:
        marks (float): The marks obtained by the student.
        Returns:
        str: The grade corresponding to the marks.
        """
        if marks >= 90:
            return "Grade A"
        elif marks >= 75:
            return "Grade B"
        else:
            return "Grade C"
grade_calculator = GradeCalculator()
marks_list = [85, 72]
for marks in marks_list:
```

```
    grade = grade_calculator.calculate_grade(marks)
    print(grade)
```

Grade B

Grade C

Grade B

Grade C

---3.py done---

Task Description #4 (Refactoring – Converting Procedural Code to Functions)

• Task: Use AI to refactor procedural input–processing logic into functions.

Instructions:

o Identify input, processing, and output sections.

o Convert each into a separate function.

o Improve code readability without changing behavior.

• Sample Legacy Code:

num = int(input("Enter number: "))
square = num * num
print("Square:", square)

• Expected Output:

o Modular code using functions like get_input(), calculate_square(), and display_result().

```
"""
num = int(input("Enter number: "))
square = num * num
print("Square:", square)
"""
"""
Refactor the procedural code by separating input, processing, and output into
functions:
get_input(), calculate_square(num), and display_result(result).
Improve readability while keeping the same behavior.
add docstrings to each function to explain their purpose and parameters.
"""
def get_input():
    """
    Get a number input from the user.
    Returns:
    int: The number entered by the user.
    """
    return int(input("Enter number: "))
def calculate_square(num):
    """
    Calculate the square of a number.
    Parameters:
    num (int): The number for which to calculate the square.
    Returns:
    int: The square of the number.
    """
    return num * num
def display_result(result):
    """
    Display the result.
    Parameters:
```

```
    result (int): The result to display.
    """
    print("Square:", result)
number = get_input()
square_result = calculate_square(number)
display_result(square_result)
```

Enter number: 7

Square: 49

---4.py done---

Task 5 (Refactoring Procedural Code into OOP Design)
• Task: Use AI to refactor procedural code into a class-based design.
Focus Areas:
o Object-Oriented principles
o Encapsulation
Legacy Code:
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
Expected Outcome:
o A class like EmployeeSalaryCalculator with methods and attributes.

```
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
"""
Refactor the procedural salary calculation code into an object-oriented design
using
a class named EmployeeSalaryCalculator with attributes and methods
to compute tax and net salary, following encapsulation principles.
"""
class EmployeeSalaryCalculator:
    """A class to calculate the net salary of an employee after tax deduction.
    """
    def __init__(self, salary):
        """
        Initialize the EmployeeSalaryCalculator with the given salary.
        Parameters:
        salary (float): The gross salary of the employee.
        """
        self.salary = salary
    def calculate_tax(self):
        """
        Calculate the tax based on the salary.
        Returns:
        float: The calculated tax amount.
        """
        return self.salary * 0.2
    def calculate_net_salary(self):
        """
        Calculate the net salary after deducting tax from the gross salary.
        Returns:
        float: The net salary of the employee.
        """
        tax = self.calculate_tax()
```

```
        net_salary = self.salary - tax
        return net_salary
employee_salary_calculator = EmployeeSalaryCalculator(50000)
net_salary = employee_salary_calculator.calculate_net_salary()
print(net_salary)
```

40000.0

Task 6 (Optimizing Search Logic)
• Task: Refactor inefficient linear searches using appropriate data structures.
• Focus Areas:
o Time complexity
o Data structure choice
Legacy Code:
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
if u == name:
found = True
print("Access Granted" if found else "Access Denied")
Expected Outcome:
o Use of sets or dictionaries with complexity justification

```
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
    if u == name:
        found = True
print("Access Granted" if found else "Access Denied")
"""
Refactor the linear search logic by replacing the list with a more efficient
data structure
such as a set or dictionary to improve lookup time and reduce time complexity
while keeping the same access check behavior.
"""
users = {"admin", "guest", "editor", "viewer"}
name = input("Enter username: ")
print("Access Granted" if name in users else "Access Denied")
```

Enter username: admin

Access Granted

Task 7 – Refactoring the Library Management System

Problem Statement

You are provided with a poorly structured Library Management script that:

- Contains repeated conditional logic
- Does not use reusable functions
- Lacks documentation
- Uses print-based procedural execution
- Does not follow modular programming principles

Your task is to refactor the code into a proper format

1. Create a module library.py with functions:
   - add_book(title, author, isbn)
   - remove_book(isbn)
   - search_book(isbn)
2. Insert triple quotes under each function and let Copilot complete the docstrings.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format.
5. Open the file in a browser.

Given Code

```
# Library Management System (Unstructured Version)
# This code needs refactoring into a proper module with documentation.
library_db = {}
# Adding first book
title = "Python Basics"
author = "John Doe"
isbn = "101"
if isbn not in library_db:
library_db[isbn] = {"title": title, "author": author}
print("Book added successfully.")
else:
print("Book already exists.")
# Adding second book (duplicate logic)
title = "AI Fundamentals"
author = "Jane Smith"
isbn = "102"
if isbn not in library_db:
library_db[isbn] = {"title": title, "author": author}
print("Book added successfully.")
else:
print("Book already exists.")
# Searching book (repeated logic structure)
isbn = "101"

if isbn in library_db:
print("Book Found:", library_db[isbn])
else:
print("Book not found.")
# Removing book (again repeated pattern)
```

```
isbn = "101"
if isbn in library_db:
del library_db[isbn]
print("Book removed successfully.")
else:
print("Book not found.")
# Searching again
isbn = "101"
if isbn in library_db:
print("Book Found:", library_db[isbn])
else:
print("Book not found.")
```

```
'''# Library Management System (Unstructured Version)
# This code needs refactoring into a proper module with documentation.
library_db = {}
# Adding first book
title = "Python Basics"
author = "John Doe"
isbn = "101"
if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")
# Adding second book (duplicate logic)
title = "AI Fundamentals"
author = "Jane Smith"
isbn = "102"
if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")
# Searching book (repeated logic structure)
isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")
# Removing book (again repeated pattern)
isbn = "101"
if isbn in library_db:
    del library_db[isbn]
    print("Book removed successfully.")
else:
    print("Book not found.")
# Searching again
```

```
isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")'''

"""
Refactor the unstructured Library Management script into a modular Python
module named library.py
by creating reusable functions add_book(title, author, isbn),
remove_book(isbn),
and search_book(isbn). Eliminate duplicate logic, add proper docstrings,
and improve code readability and maintainability.
"""
class Library:
    """A class to represent a library management system."""
    def __init__(self):
        """Initialize the library database."""
        self.library_db = {}
    def add_book(self, title, author, isbn):
        """
        Add a book to the library database.
        Parameters:
        title (str): The title of the book.
        author (str): The author of the book.
        isbn (str): The ISBN number of the book.
        """
        if isbn not in self.library_db:
            self.library_db[isbn] = {"title": title, "author": author}
            print("Book added successfully.")
        else:
            print("Book already exists.")
    def remove_book(self, isbn):
        """
        Remove a book from the library database.
        Parameters:
        isbn (str): The ISBN number of the book to be removed.
        """
        if isbn in self.library_db:
            del self.library_db[isbn]
            print("Book removed successfully.")
        else:
            print("Book not found.")
    def search_book(self, isbn):
        """
        Search for a book in the library database.
        Parameters:
        isbn (str): The ISBN number of the book to search for.
```

```
        Returns:
        dict: The details of the book if found, otherwise None.
        """
        if isbn in self.library_db:
            return self.library_db[isbn]
        else:
            print("Book not found.")
            return None
library = Library()
library.add_book("Python Basics", "John Doe", "101")
library.add_book("AI Fundamentals", "Jane Smith", "102")
print(library.search_book("101"))
library.remove_book("101")
print(library.search_book("101"))
```

Book added successfully.

Book added successfully.

{'title': 'Python Basics', 'author': 'John Doe'}

Book removed successfully.

Book not found.

None

---7.py done---

Task 8– Fibonacci Generator

Write a program to generate Fibonacci series up to n.

The initial code has:

• Global variables.

• Inefficient loop.

• No functions or modularity.

Task for Students:

• Refactor into a clean reusable function (generate_fibonacci).

• Add docstrings and test cases.

• Compare AI-refactored vs original.

Bad Code Version:

```
# fibonacci bad version
n=int(input("Enter limit: "))
a=0

b=1
print(a)
print(b)
for i in range(2,n):
c=a+b
print(c)
a=b
b=c
```

```
"""# fibonacci bad version
n=int(input("Enter limit: "))
a=0
b=1
print(a)
print(b)
for i in range(2,n):
    c=a+b
    print(c)
    a=b
    b=c"""
"""
Refactor the Fibonacci program into a reusable function generate_fibonacci(n), remove global variables, improve loop logic, add docstrings, and keep the same output behavior.
"""
def generate_fibonacci(n):
    """
    Generate Fibonacci numbers up to the nth number.
    Parameters:
    n (int): The number of Fibonacci numbers to generate.
    Returns:
    list: A list of Fibonacci numbers up to the nth number.
    """
```

```
    if n <= 0:
        return []
    elif n == 1:
        return [0]
    elif n == 2:
        return [0, 1]

    fib_sequence = [0, 1]
    for i in range(2, n):
        next_fib = fib_sequence[i-1] + fib_sequence[i-2]
        fib_sequence.append(next_fib)

    return fib_sequence
limit = int(input("Enter limit: "))
fibonacci_numbers = generate_fibonacci(limit)
for num in fibonacci_numbers:
    print(num)
```

Enter limit: 8

0

1

1

2

3

5

8

13

---8.py done---

Task 9 – Twin Primes Checker
Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).
The initial code has:
• Inefficient prime checking.
• No functions.
• Hardcoded inputs.
Task for Students:
• Refactor into is_prime(n) and is_twin_prime(p1, p2).
• Add docstrings and optimize.
• Generate a list of twin primes in a given range using AI.
Bad Code Version:
# twin primes bad version
a=11
b=13
fa=0
for i in range(2,a):
if a%i==0:
fa=1
fb=0
for i in range(2,b):
if b%i==0:
fb=1
if fa==0 and fb==0 and abs(a-b)==2:
print("Twin Primes")
else:
print("Not Twin Primes")

```
"""# twin primes bad version
a=11
b=13
fa=0
for i in range(2,a):
    if a%i==0:
        fa=1

fb=0
for i in range(2,b):
    if b%i==0:
        fb=1

if fa==0 and fb==0 and abs(a-b)==2:
    print("Twin Primes")
else:
    print("Not Twin Primes")"""
"""
Refactor the twin primes code by creating functions is_prime(n) and
is_twin_prime(p1, p2),
```

```
optimize prime checking, add docstrings, and allow generating twin primes in a
range.
"""
def is_prime(n):
    """Check if a number is prime.
    Parameters:
    n (int): The number to check for primality.
    Returns:
    bool: True if the number is prime, False otherwise.
    """
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True

def is_twin_prime(p1, p2):
    """Check if two numbers are twin primes.
    Parameters:
    p1 (int): The first number.
    p2 (int): The second number.
    Returns:
    bool: True if the numbers are twin primes, False otherwise.
    """
    return is_prime(p1) and is_prime(p2) and abs(p1 - p2) == 2
def generate_twin_primes(start, end):
    """Generate twin primes within a specified range.
    Parameters:
    start (int): The starting number of the range.
    end (int): The ending number of the range.
    Returns:
    list: A list of tuples, each containing a pair of twin primes.
    """
    twin_primes = []
    for num in range(start, end - 1):
        if is_twin_prime(num, num + 2):
            twin_primes.append((num, num + 2))
    return twin_primes
start_range = int(input("Enter start of range: "))
end_range = int(input("Enter end of range: "))
twin_prime_pairs = generate_twin_primes(start_range, end_range)
```

```
if twin_prime_pairs:
    print("Twin Primes in the range:")
    for pair in twin_prime_pairs:
        print(pair)
else:    print("No twin primes found in the range.")
```

Enter start of range: 1

Enter end of range: 20

Twin Primes in the range:

(3, 5)

(5, 7)

(11, 13)

(17, 19)

---9.py done---

Task 10 – Refactoring the Chinese Zodiac Program

Objective

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

The current program reads a year from the user and prints the corresponding Chinese Zodiac sign. However, the implementation contains repetitive

conditional logic, lacks modular design, and does not follow clean coding principles.

Your task is to refactor the code to improve readability, maintainability, and structure.

Chinese Zodiac Cycle (Repeats Every 12 Years)

1. Rat
2. Ox
3. Tiger
4. Rabbit
5. Dragon
6. Snake
7. Horse
8. Goat (Sheep)
9. Monkey
10. Rooster
11. Dog
12. Pig

```
# Chinese Zodiac Program (Unstructured Version)
# This code needs refactoring.
year = int(input("Enter a year: "))
if year % 12 == 0:
print("Monkey")
elif year % 12 == 1:
print("Rooster")
elif year % 12 == 2:
print("Dog")
elif year % 12 == 3:
print("Pig")
elif year % 12 == 4:
print("Rat")
elif year % 12 == 5:
print("Ox")
elif year % 12 == 6:
print("Tiger")
elif year % 12 == 7:
print("Rabbit")
elif year % 12 == 8:
print("Dragon")

elif year % 12 == 9:
print("Snake")
elif year % 12 == 10:
```

print("Horse")
elif year % 12 == 11:
print("Goat")
You must:
1. Create a reusable function: get_zodiac(year)
2. Replace the if-elif chain with a cleaner structure (e.g., list or dictionary).
3. Add proper docstrings.
4. Separate input handling from logic.
5. Improve readability and maintainability.
6. Ensure output remains correct.

```
"""# Chinese Zodiac Program (Unstructured Version)
# This code needs refactoring.

year = int(input("Enter a year: "))

if year % 12 == 0:
    print("Monkey")
elif year % 12 == 1:
    print("Rooster")
elif year % 12 == 2:
    print("Dog")
elif year % 12 == 3:
    print("Pig")
elif year % 12 == 4:
    print("Rat")
elif year % 12 == 5:
    print("Ox")
elif year % 12 == 6:
    print("Tiger")
elif year % 12 == 7:
    print("Rabbit")
elif year % 12 == 8:
    print("Dragon")
elif year % 12 == 9:
    print("Snake")
elif year % 12 == 10:
    print("Horse")
elif year % 12 == 11:
    print("Goat")"""

"""
Refactor the Chinese Zodiac program by creating a reusable function
get_zodiac(year),
replace the if-elif chain with a cleaner structure like a list or dictionary,
add docstrings, and separate input from logic.
"""
```

```python
def get_zodiac(year):
    """Return the Chinese Zodiac sign for a given year.
    Parameters:
    year (int): The year for which to determine the zodiac sign.
    Returns:
    str: The Chinese Zodiac sign corresponding to the given year.
    """
    zodiac_signs = [
        "Monkey", "Rooster", "Dog", "Pig", "Rat", "Ox",
        "Tiger", "Rabbit", "Dragon", "Snake", "Horse", "Goat"
    ]
    return zodiac_signs[year % 12]
year_input = int(input("Enter a year: "))
zodiac = get_zodiac(year_input)
print(zodiac)
```

Enter a year: 2024

Dragon

---10.py done---

Task 11 – Refactoring the Harshad (Niven) Number Checker
Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.
A Harshad (Niven) number is a number that is divisible by the sum of its digits.
For example:
• 18 → 1 + 8 = 9 → 18 ÷ 9 = 2 (Harshad Number)
• 19 → 1 + 9 = 10 → 19 ÷ 10 ≠ integer (Not Harshad)
Problem Statement
The current implementation:
• Mixes logic and input handling
• Uses redundant variables
• Does not use reusable functions properly
• Returns print statements instead of boolean values
• Lacks documentation
You must refactor the code to follow clean coding principles.
# Harshad Number Checker (Unstructured Version)
num = int(input("Enter a number: "))
temp = num
sum_digits = 0
while temp > 0:

digit = temp % 10
sum_digits = sum_digits + digit
temp = temp // 10
if sum_digits != 0:
if num % sum_digits == 0:
print("True")
else:
print("False")
else:
print("False")
You must:
1. Create a reusable function: is_harshad(number)
2. The function must:
o Accept an integer parameter.
o Return True if the number is divisible by the sum of its digits.
o Return False otherwise.
3. Separate user input from core logic.
4. Add proper docstrings.
5. Improve readability and maintainability.
6. Ensure the program handles edge cases (e.g., 0, negative numbers).

```
"""# Harshad Number Checker (Unstructured Version)

num = int(input("Enter a number: "))

temp = num
```

```
sum_digits = 0

while temp > 0:
    digit = temp % 10
    sum_digits = sum_digits + digit
    temp = temp // 10

if sum_digits != 0:
    if num % sum_digits == 0:
        print("True")
    else:
        print("False")
else:
    print("False")"""

"""
Refactor the Harshad number checker by creating a reusable function
is_harshad(number) that returns True or False, separate input from logic,
add docstrings, and handle edge cases.
"""
def is_harshad(number):
    """Check if a number is a Harshad number.
    A Harshad number is an integer that is divisible by the sum of its digits.
    Parameters:
    number (int): The number to check.
    Returns:
    bool: True if the number is a Harshad number, False otherwise.
    """
    if number <= 0:
        return False

    temp = number
    sum_digits = 0

    while temp > 0:
        digit = temp % 10
        sum_digits += digit
        temp //= 10

    return sum_digits != 0 and number % sum_digits == 0
user_input = int(input("Enter a number: "))
if is_harshad(user_input):
    print("True")
else:
    print("False")
```

Enter a number: 18

True

---11.py done---

Task 12 – Refactoring the Factorial Trailing Zeros Program
Refactor the given poorly structured Python script into a clean, modular, and efficient implementation.
The program calculates the number of trailing zeros in n! (factorial of n).
Problem Statement
The current implementation:
• Calculates the full factorial (inefficient for large n)
• Mixes input handling with business logic
• Uses print statements instead of return values
• Lacks modular structure and documentation
You must refactor the code to improve efficiency, readability, and maintainability.

# Factorial Trailing Zeros (Unstructured Version)
n = int(input("Enter a number: "))
fact = 1
i = 1
while i <= n:
fact = fact * i
i = i + 1
count = 0
while fact % 10 == 0:
count = count + 1
fact = fact // 10
print("Trailing zeros:", count)
You must:
1. Create a reusable function: count_trailing_zeros(n)
2. The function must:
o Accept a non-negative integer n.
o Return the number of trailing zeros in n!.
3. Do NOT compute the full factorial.
4. Use an optimized mathematical approach (count multiples of 5).
5. Add proper docstrings.
6. Separate user interaction from core logic.
7. Handle edge cases (e.g., negative numbers, zero).

```
"""# Factorial Trailing Zeros (Unstructured Version)

n = int(input("Enter a number: "))

fact = 1
i = 1

while i <= n:
    fact = fact * i
    i = i + 1

count = 0
```

```
while fact % 10 == 0:
    count = count + 1
    fact = fact // 10

print("Trailing zeros:", count)"""

"""
Refactor the trailing zeros program by creating a
function count_trailing_zeros(n) using the optimized multiples-of-5 method,
avoid computing factorial, add docstrings, and separate input from logic.
"""
def count_trailing_zeros(n):
    """Count the number of trailing zeros in n! (n factorial).
    This is done by counting the number of times 5 is a factor in the numbers
from 1 to n.
    Parameters:
    n (int): The number for which to count trailing zeros in its factorial.
    Returns:
    int: The number of trailing zeros in n!.
    """
    count = 0
    power_of_5 = 5
    while n >= power_of_5:
        count += n // power_of_5
        power_of_5 *= 5
    return count
number = int(input("Enter a number: "))
print("Trailing zeros:", count_trailing_zeros(number))
```

Enter a number: 25

Trailing zeros: 6

---12.py done---

Task Description #13: Smart Airport Management System – Data Structure Challenge

An airport authority plans to implement:

1. Passenger Check-In Queue – Process passengers in order of arrival.
2. Emergency Landing Priority – Aircraft in emergency get priority.
3. Flight Information Lookup – Fast search using Flight ID.
4. Air Route Connectivity – Airports connected via routes.
5. Boarding Process (Last-In First-Out Cabin Storage) – Overhead luggage management simulation.

Student Task:

- Choose suitable data structures from the provided list.
- Provide 2–3 sentence justification per feature.
- Implement one feature in Python with comments and docstrings using AI assistance.

Expected Output:

- Mapping table.
- Working Python implementation.

13.1

```
"""
Smart Airport Management System

Feature: Passenger Check-In Queue

Passengers must be processed in the order they arrive at the airport.
Design a Python implementation using an appropriate data structure that
follows FIFO (First-In First-Out) behavior.

Tasks:
- Choose the correct data structure.
- Implement passenger check-in and processing logic in Python.
- Add comments and docstrings.
"""
from collections import deque
class AirportCheckIn:
    """A class to manage the passenger check-in queue at the airport."""
    def __init__(self):
        """Initialize the check-in queue using a deque for efficient FIFO
operations."""
        self.check_in_queue = deque()
    def check_in_passenger(self, passenger_name):
        """
        Add a passenger to the check-in queue.
        Parameters:
        passenger_name (str): The name of the passenger to be checked in.
        """
```

```
        self.check_in_queue.append(passenger_name)
        print(f"{passenger_name} has been checked in.")
    def process_passenger(self):
        """
        Process the next passenger in the check-in queue.
        Returns:
        str: The name of the processed passenger or a message if the queue is
empty.
        """
        if self.check_in_queue:
            passenger_name = self.check_in_queue.popleft()
            print(f"{passenger_name} has been processed.")
            return passenger_name
        else:
            print("No passengers in the queue.")
            return None
    def get_queue_length(self):
        """
        Get the current number of passengers in the check-in queue.
        Returns:
        int: The number of passengers waiting in the queue.
        """
        return len(self.check_in_queue)
# Example usage
airport_check_in = AirportCheckIn()
airport_check_in.check_in_passenger("Alice")
airport_check_in.check_in_passenger("Bob")
airport_check_in.process_passenger()
airport_check_in.process_passenger()
airport_check_in.check_in_passenger("Charlie")
print(airport_check_in.get_queue_length())
```

13.2

```
"""
Smart Airport Management System

Feature: Emergency Landing Priority

Aircraft requesting emergency landing must be given higher priority
than normal aircraft waiting for landing.

Tasks:
- Select a suitable data structure that supports priority handling.
- Implement a Python simulation where emergency aircraft are handled first.
- Include comments and docstrings.
"""
```

```
from collections import deque
class AirportLanding:
    """A class to manage the landing queue at the airport, prioritizing
emergency landings."""
    def __init__(self):
        """Initialize the landing queue using a deque for efficient FIFO
operations."""
        self.landing_queue = deque()
    def request_landing(self, aircraft_name, is_emergency=False):
        """
        Add an aircraft to the landing queue with priority for emergencies.
        Parameters:
        aircraft_name (str): The name of the aircraft requesting landing.
        is_emergency (bool): Whether the landing request is an emergency.
        """
        if is_emergency:
            self.landing_queue.appendleft(aircraft_name)
            print(f"{aircraft_name} has requested an emergency landing.")
        else:
            self.landing_queue.append(aircraft_name)
            print(f"{aircraft_name} has requested a normal landing.")
    def process_landing(self):
        """
        Process the next aircraft in the landing queue.
        Returns:
        str: The name of the processed aircraft or a message if the queue is
empty.
        """
        if self.landing_queue:
            aircraft_name = self.landing_queue.popleft()
            print(f"{aircraft_name} has landed.")
            return aircraft_name
        else:
            print("No aircraft in the landing queue.")
            return None
# Example usage
airport_landing = AirportLanding()
airport_landing.request_landing("Flight 101")
airport_landing.request_landing("Flight 202", is_emergency=True)
airport_landing.request_landing("Flight 303")
airport_landing.process_landing()
airport_landing.process_landing()
airport_landing.process_landing()
```

13.3

```
"""
Smart Airport Management System
```

```
Feature: Flight Information Lookup

Airport staff must quickly retrieve flight details using a Flight ID.

Tasks:
- Choose a data structure that allows fast lookup by key.
- Implement a Python program to store and retrieve flight information.
- Include comments and docstrings.
"""
class FlightInfo:
    """A class to manage flight information in the airport."""
    def __init__(self):
        """Initialize the flight information database."""
        self.flight_db = {}
    def add_flight(self, flight_id, destination, departure_time):
        """
        Add a flight to the flight information database.
        Parameters:
        flight_id (str): The unique identifier for the flight.
        destination (str): The destination of the flight.
        departure_time (str): The departure time of the flight.
        """
        self.flight_db[flight_id] = {
            "destination": destination,
            "departure_time": departure_time
        }
        print(f"Flight {flight_id} added successfully.")
    def get_flight_info(self, flight_id):
        """
        Retrieve flight information by Flight ID.
        Parameters:
        flight_id (str): The unique identifier for the flight to look up.
        Returns:
        dict: A dictionary containing the flight's destination and departure
time,
              or a message if the flight is not found.
        """
        if flight_id in self.flight_db:
            return self.flight_db[flight_id]
        else:
            return "Flight not found."
# Example usage
flight_info = FlightInfo()
flight_info.add_flight("FL123", "New York", "10:00 AM")
flight_info.add_flight("FL456", "Los Angeles", "2:00 PM")
print(flight_info.get_flight_info("FL123"))
print(flight_info.get_flight_info("FL789"))
```

13.4

```
"""
Smart Airport Management System

Feature: Air Route Connectivity

Airports are connected through different routes.
We must represent connections between airports efficiently.

Tasks:
- Choose a suitable data structure for representing network connections.
- Implement a Python representation of airport connections.
- Include comments and docstrings.
"""
class AirportNetwork:
    """A class to represent the network of airports and their connections."""
    def __init__(self):
        """Initialize the airport network using an adjacency list."""
        self.network = {}
    def add_connection(self, airport1, airport2):
        """
        Add a bidirectional connection between two airports.
        Parameters:
        airport1 (str): The name of the first airport.
        airport2 (str): The name of the second airport.
        """
        if airport1 not in self.network:
            self.network[airport1] = []
        if airport2 not in self.network:
            self.network[airport2] = []
        self.network[airport1].append(airport2)
        self.network[airport2].append(airport1)
    def get_connections(self, airport):
        """
        Retrieve the list of connected airports for a given airport.
        Parameters:
        airport (str): The name of the airport to look up.
        Returns:
        list: A list of connected airports or a message if the airport is not
found.
        """
        if airport in self.network:
            return self.network[airport]
        else:
            return "Airport not found."
```

```
# Example usage
airport_network = AirportNetwork()
airport_network.add_connection("JFK", "LAX")
airport_network.add_connection("JFK", "ORD")
airport_network.add_connection("LAX", "ORD")
print(airport_network.get_connections("JFK"))
print(airport_network.get_connections("LAX"))
print(airport_network.get_connections("ORD"))
```

13.5

```
"""
Smart Airport Management System

Feature: Boarding Cabin Luggage Storage

Passengers store luggage in overhead bins where the last placed bag
is removed first.

Tasks:
- Choose a data structure suitable for Last-In First-Out behavior.
- Implement luggage storage and removal simulation in Python.
- Include comments and docstrings.
"""
class LuggageStorage:
    """A class to represent the luggage storage system in an airplane
cabin."""
    def __init__(self):
        """Initialize the luggage storage using a list to simulate a stack."""
        self.storage = []
    def store_luggage(self, luggage_item):
        """
        Store a luggage item in the overhead bin.
        Parameters:
        luggage_item (str): The description of the luggage item to be stored.
        """
        self.storage.append(luggage_item)
        print(f"Luggage '{luggage_item}' stored successfully.")
    def remove_luggage(self):
        """
        Remove the last stored luggage item from the overhead bin.
        Returns:
        str: The description of the removed luggage item or a message if
storage is empty.
        """
        if self.storage:
            removed_item = self.storage.pop()
            print(f"Luggage '{removed_item}' removed successfully.")
```

```
            return removed_item
        else:
            print("No luggage to remove.")
            return None
# Example usage
luggage_storage = LuggageStorage()
luggage_storage.store_luggage("Suitcase A")
luggage_storage.store_luggage("Backpack B")
luggage_storage.remove_luggage()
luggage_storage.remove_luggage()
```

Alice has been checked in.

Bob has been checked in.

Alice has been processed.

Bob has been processed.

Charlie has been checked in.

1

---13.1 done---

Flight 101 has requested a normal landing.

Flight 202 has requested an emergency landing.

Flight 303 has requested a normal landing.

Flight 202 has landed.

Flight 101 has landed.

Flight 303 has landed.

---13.2 done---

Flight FL123 added successfully.

Flight FL456 added successfully.

{'destination': 'New York', 'departure_time': '10:00 AM'}

Flight not found.

---13.3 done---

['LAX', 'ORD']

['JFK', 'ORD']

['JFK', 'LAX']

---13.4 done---

Luggage 'Suitcase A' stored successfully.

Luggage 'Backpack B' stored successfully.

Luggage 'Backpack B' removed successfully.

Luggage 'Suitcase A' removed successfully.

---13.5 done---

Feature	Data Structure	Justification
Passenger Check-In Queue	Queue (Deque)	Passengers must be processed in the order they arrive — FIFO behavior. Deque provides O(1) append and popleft operations, making it ideal for queue simulation.
Emergency Landing Priority	Deque (Priority Queue)	Emergency aircraft must jump ahead of normal ones. Deque's appendleft() allows inserting emergency requests at the front in O(1) time.
Flight Information Lookup	Hash Table (Dictionary)	Flight details must be retrieved instantly using a Flight ID as the key. Dictionary provides O(1) average-case lookup, which is optimal for key-based search.
Air Route Connectivity	Graph (Adjacency List)	Airports are interconnected nodes with bidirectional routes as edges. An adjacency list (dictionary of lists) efficiently represents sparse networks and supports traversal algorithms.
Boarding Cabin Luggage Storage	Stack (List)	Luggage placed last in the overhead bin is removed first — LIFO behavior. Python list with append() and pop() provides O(1) stack operations.

Task Description #14: Smart Social Media Platform – Data Structure Selection
A social media company wants to develop a system that includes:
1. News Feed Generation – Display posts chronologically.
2. Trending Hashtags Tracker – Rank hashtags by usage frequency.
3. User Profile Search – Quick lookup using username.
4. Friend Network Representation – Represent user connections.
5. Message Undo Feature – Allow last sent message to be withdrawn.
Student Task:
• Select appropriate data structures from the list.
• Justify your choices clearly.
• Implement one selected feature as a Python program using AI-assisted code generation.
Expected Output:
• Table mapping features to data structures with justification.
• Working Python code with docstrings and comments.

14.1

```
"""
Smart Social Media Platform

Feature: News Feed Generation

Posts should appear in chronological order in the user's news feed.

Tasks:
- Choose an appropriate data structure to maintain chronological order.
- Implement a Python simulation for storing and displaying posts.
- Include comments and docstrings.
"""
from collections import deque
class NewsFeed:
    """A class to represent a user's news feed on a social media platform."""
    def __init__(self):
        """Initialize the news feed using a deque to maintain chronological order."""
        self.posts = deque()
    def add_post(self, post_content):
        """
        Add a new post to the news feed.
        Parameters:
        post_content (str): The content of the post to be added.
        """
        self.posts.append(post_content)
        print(f"Post added: {post_content}")
    def display_feed(self):
        """Display all posts in the news feed in chronological order."""
        print("News Feed:")
```

```
        for post in self.posts:
            print(post)
# Example usage
news_feed = NewsFeed()
news_feed.add_post("Hello, world!")
news_feed.add_post("Welcome to my news feed.")
news_feed.display_feed()
```

14.2

```
"""
Smart Social Media Platform

Feature: Trending Hashtags Tracker

The system must track hashtags and identify which ones are used most often.

Tasks:
- Choose a data structure suitable for tracking hashtag frequency.
- Implement a Python program to store hashtags and identify trending ones.
- Include comments and docstrings.
"""
class HashtagTracker:
    """A class to track hashtags and identify trending ones."""
    def __init__(self):
        """Initialize the hashtag tracker using a dictionary to count
frequencies."""
        self.hashtag_counts = {}
    def add_hashtag(self, hashtag):
        """
        Add a hashtag to the tracker and update its count.
        Parameters:
        hashtag (str): The hashtag to be added.
        """
        if hashtag in self.hashtag_counts:
            self.hashtag_counts[hashtag] += 1
        else:
            self.hashtag_counts[hashtag] = 1
        print(f"Hashtag '{hashtag}' added. Current count:
{self.hashtag_counts[hashtag]}")
    def get_trending_hashtags(self, top_n=5):
        """
        Retrieve the top N trending hashtags based on their counts.
        Parameters:
        top_n (int): The number of top trending hashtags to return.
        Returns:
        list: A list of the top N trending hashtags sorted by frequency.
```

```
        """
        sorted_hashtags = sorted(self.hashtag_counts.items(), key=lambda item:
item[1], reverse=True)
        return [hashtag for hashtag, count in sorted_hashtags[:top_n]]
# Example usage
hashtag_tracker = HashtagTracker()
hashtag_tracker.add_hashtag("#AI")
hashtag_tracker.add_hashtag("#Python")
print(hashtag_tracker.get_trending_hashtags())
```

14.3

```
"""
Smart Social Media Platform

Feature: User Profile Search

The system must allow quick lookup of user profiles using usernames.

Tasks:
- Choose a data structure that allows fast searching by username.
- Implement a Python program to store and retrieve user profiles.
- Include comments and docstrings.
"""

class UserProfile:
    """A class to represent a user profile in the social media platform."""
    def __init__(self):
        """Initialize the user profile database."""
        self.user_db = {}
    def add_user(self, username, full_name, bio):
        """
        Add a user profile to the database.
        Parameters:
        username (str): The unique username for the user.
        full_name (str): The full name of the user.
        bio (str): A short biography of the user.
        """
        self.user_db[username] = {
            "full_name": full_name,
            "bio": bio
        }
        print(f"User '{username}' added successfully.")
    def get_user_profile(self, username):
        """
        Retrieve a user profile by username.
        Parameters:
        username (str): The unique username to look up.
```

```
        Returns:
        dict: A dictionary containing the user's full name and bio,
              or a message if the user is not found.
        """
        if username in self.user_db:
            return self.user_db[username]
        else:
            return "User not found."
# Example usage
user_profile = UserProfile()
user_profile.add_user("johndoe", "John Doe", "Software developer and tech
enthusiast.")
print(user_profile.get_user_profile("johndoe"))
print(user_profile.get_user_profile("janedoe"))
```

14.4

```
"""
Smart Social Media Platform

Feature: Friend Network Representation

Users can connect with other users, forming a network of friendships.

Tasks:
- Choose a data structure suitable for representing relationships between
users.
- Implement a Python program to represent and display the friend network.
- Include comments and docstrings.
"""
class FriendNetwork:
    """A class to represent a social media platform's friend network."""
    def __init__(self):
        """Initialize the friend network using an adjacency list."""
        self.network = {}
    def add_friend(self, user1, user2):
        """
        Add a friendship connection between two users.
        Parameters:
        user1 (str): The name of the first user.
        user2 (str): The name of the second user.
        """
        if user1 not in self.network:
            self.network[user1] = []
        if user2 not in self.network:
            self.network[user2] = []
        self.network[user1].append(user2)
```

```
        self.network[user2].append(user1)
    def display_network(self):
        """Display the friend network."""
        for user, friends in self.network.items():
            print(f"{user} is friends with: {', '.join(friends)}")
# Example usage
friend_network = FriendNetwork()
friend_network.add_friend("Alice", "Bob")
friend_network.add_friend("Alice", "Charlie")
friend_network.add_friend("Bob", "David")
friend_network.display_network()
```

14.5

```
"""
Smart Social Media Platform

Feature: Message Undo

Users should be able to undo the last message they sent.

Tasks:
- Choose a suitable data structure to support undo functionality.
- Implement a Python simulation where the last message can be removed.
- Include comments and docstrings.
"""
class MessageHistory:
    """A class to represent the message history and support undo
functionality."""
    def __init__(self):
        """Initialize the message history using a list to store messages."""
        self.messages = []
    def send_message(self, message):
        """
        Send a new message and add it to the history.
        Parameters:
        message (str): The content of the message to be sent.
        """
        self.messages.append(message)
        print(f"Message sent: {message}")
    def undo_message(self):
        """
        Undo the last sent message by removing it from the history.
        Returns:
        str: The content of the undone message or a message if there are no
messages to undo.
        """
        if self.messages:
```

```
            undone_message = self.messages.pop()
            print(f"Message undone: {undone_message}")
            return undone_message
        else:
            print("No messages to undo.")
            return None
# Example usage
message_history = MessageHistory()
message_history.send_message("Hello, world!")
message_history.send_message("Welcome to my message history.")
message_history.undo_message()
message_history.undo_message()
message_history.undo_message()
```

Post added: Hello, world!

Post added: Welcome to my news feed.

News Feed:

Hello, world!

Welcome to my news feed.

---14.1 done---

Hashtag '#AI' added. Current count: 1

Hashtag '#Python' added. Current count: 1

['#AI', '#Python']

---14.2 done---

User 'johndoe' added successfully.

{'full_name': 'John Doe', 'bio': 'Software developer and tech enthusiast.'}

User not found.

---14.3 done---

Alice is friends with: Bob, Charlie

Bob is friends with: Alice, David

Charlie is friends with: Alice

David is friends with: Bob

---14.4 done---

Message sent: Hello, world!

Message sent: Welcome to my message history.

Message undone: Welcome to my message history.

Message undone: Hello, world!

No messages to undo.

---14.5 done---

Feature	Data Structure	Justification
News Feed Generation	Queue (Deque)	Posts must appear in chronological order — FIFO. Deque supports efficient append at the rear and iteration in insertion order, maintaining the timeline correctly.
Trending Hashtags Tracker	Hash Table (Dictionary)	Hashtag frequency must be tracked and sorted by count. A dictionary maps each hashtag to its count in O(1) time, and sorting by value identifies trending ones efficiently.
User Profile Search	Hash Table (Dictionary)	Usernames serve as unique keys for quick profile retrieval. Dictionary provides O(1) average-case lookup, far faster than linear search through a list.
Friend Network Representation	Graph (Adjacency List)	Users are nodes and friendships are bidirectional edges. An adjacency list (dictionary of lists) efficiently represents the sparse social graph and supports connection queries.
Message Undo Feature	Stack (List)	The last sent message must be withdrawn first — LIFO behavior. Python list with append() and pop() implements a stack perfectly for an undo history.

Task #15: Smart University Hostel Management
Scenario:
A university wants to build a hostel management system that manages:
1. Room Allotment
2. Medical Emergency Handling
3. Student Record Retrieval
4. Hostel Block Connectivity
5. Visitor Entry Log
Student Task
• For each feature, select the most appropriate data structure from the list

below:
o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque
• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.
Expected Output:
• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.

15.1

```
"""
Smart University Hostel Management System

Feature: Room Allotment

Students must be assigned rooms in the hostel.

Tasks:
- Choose an appropriate data structure to manage room allotments.
- Implement a Python program to assign and manage rooms.
- Include comments and docstrings.
"""
class HostelManagement:
    """A class to manage room allotments in a university hostel."""
    def __init__(self):
        """Initialize the hostel management system with an empty room
allotment."""
        self.rooms = {}
```

```
    def allot_room(self, student_name, room_number):
        """
        Allot a room to a student.
        Parameters:
        student_name (str): The name of the student to be allotted a room.
        room_number (str): The number of the room to be allotted.
        """
        if room_number in self.rooms:
            print(f"Room {room_number} is already occupied by
{self.rooms[room_number]}.")
        else:
            self.rooms[room_number] = student_name
            print(f"Room {room_number} allotted to {student_name}.")
    def get_room_allotment(self, room_number):
        """
        Retrieve the name of the student allotted to a specific room.
        Parameters:
        room_number (str): The number of the room to look up.
        Returns:
        str: The name of the student allotted to the room or a message if the
room is vacant.
        """
        if room_number in self.rooms:
            return self.rooms[room_number]
        else:
            return f"Room {room_number} is vacant."
# Example usage
hostel = HostelManagement()
hostel.allot_room("Alice", "101")
hostel.allot_room("Bob", "102")
print(hostel.get_room_allotment("101"))
print(hostel.get_room_allotment("103"))
```

15.2

```
"""
Smart University Hostel Management System

Feature: Medical Emergency Handling

Emergency medical cases must be handled with higher priority.

Tasks:
- Choose a suitable data structure to manage emergency cases.
- Implement a Python simulation for handling emergencies.
- Include comments and docstrings.
"""
class MedicalEmergencyHandler:
```

```
    """A class to manage medical emergencies in a university hostel."""
    def __init__(self):
        """Initialize the medical emergency handler using a list to simulate a
queue."""
        self.emergency_queue = []
    def add_emergency(self, case_description):
        """
        Add a medical emergency case to the queue.
        Parameters:
        case_description (str): The description of the medical emergency case.
        """
        self.emergency_queue.append(case_description)
        print(f"Medical emergency added: {case_description}")
    def handle_emergency(self):
        """
        Handle the next medical emergency case in the queue.
        Returns:
        str: The description of the handled emergency case or a message if
there are no emergencies to handle.
        """
        if self.emergency_queue:
            handled_case = self.emergency_queue.pop(0)
            print(f"Medical emergency handled: {handled_case}")
            return handled_case
        else:
            print("No medical emergencies to handle.")
            return None
# Example usage
emergency_handler = MedicalEmergencyHandler()
emergency_handler.add_emergency("Student A has a severe headache.")
emergency_handler.add_emergency("Student B has a broken arm.")
emergency_handler.handle_emergency()
emergency_handler.handle_emergency()
emergency_handler.handle_emergency()
```

15.3

```
"""
Smart University Hostel Management System

Feature: Student Record Retrieval

The system must allow quick retrieval of student records.

Tasks:
- Choose an appropriate data structure for fast student lookup.
- Implement a Python program to store and retrieve student records.
- Include comments and docstrings.
```

```
"""
class StudentRecord:
    """A class to manage student records in a university hostel."""
    def __init__(self):
        """Initialize the student record management system."""
        self.student_db = {}
    def add_student(self, student_id, name, age, course):
        """
        Add a student record to the database.
        Parameters:
        student_id (str): The unique identifier for the student.
        name (str): The name of the student.
        age (int): The age of the student.
        course (str): The course the student is enrolled in.
        """
        self.student_db[student_id] = {
            "name": name,
            "age": age,
            "course": course
        }
        print(f"Student {name} added successfully.")
    def get_student_record(self, student_id):
        """
        Retrieve a student record by Student ID.
        Parameters:
        student_id (str): The unique identifier for the student to look up.
        Returns:
        dict: A dictionary containing the student's name, age, and course,
              or a message if the student is not found.
        """
        if student_id in self.student_db:
            return self.student_db[student_id]
        else:
            return "Student not found."
# Example usage
student_record = StudentRecord()
student_record.add_student("S001", "Alice", 20, "Computer Science")
student_record.add_student("S002", "Bob", 22, "Mechanical Engineering")
print(student_record.get_student_record("S001"))
print(student_record.get_student_record("S003"))
```

15.4

```
"""
Smart University Hostel Management System

Feature: Hostel Block Connectivity
```

```
Different hostel blocks are connected and may share facilities.

Tasks:
- Choose a suitable data structure to represent block connections.
- Implement a Python representation of hostel block connectivity.
- Include comments and docstrings.
"""
class HostelBlockNetwork:
    """A class to represent the network of hostel blocks and their
connections."""
    def __init__(self):
        """Initialize the hostel block network using an adjacency list."""
        self.network = {}
    def add_connection(self, block1, block2):
        """
        Add a bidirectional connection between two hostel blocks.
        Parameters:
        block1 (str): The name of the first hostel block.
        block2 (str): The name of the second hostel block.
        """
        if block1 not in self.network:
            self.network[block1] = []
        if block2 not in self.network:
            self.network[block2] = []
        self.network[block1].append(block2)
        self.network[block2].append(block1)
    def get_connections(self, block):
        """
        Retrieve the list of connected blocks for a given hostel block.
        Parameters:
        block (str): The name of the hostel block to look up.
        Returns:
        list: A list of connected blocks or a message if the block is not
found.
        """
        if block in self.network:
            return self.network[block]
        else:
            return "Hostel block not found."
# Example usage
hostel_block_network = HostelBlockNetwork()
hostel_block_network.add_connection("Block A", "Block B")
hostel_block_network.add_connection("Block A", "Block C")
hostel_block_network.add_connection("Block B", "Block C")
print(hostel_block_network.get_connections("Block A"))
print(hostel_block_network.get_connections("Block B"))
print(hostel_block_network.get_connections("Block C"))
print(hostel_block_network.get_connections("Block D"))
```

15.5

```
"""
Smart University Hostel Management System

Feature: Visitor Entry Log

Visitors entering the hostel must be recorded and tracked.

Tasks:
- Choose a data structure suitable for maintaining visitor logs.
- Implement a Python program to add and view visitor entries.
- Include comments and docstrings.
"""
class VisitorLog:
    """A class to manage the visitor entry log for a university hostel."""
    def __init__(self):
        """Initialize the visitor log using a list to store entries."""
        self.visitor_entries = []
    def add_visitor(self, visitor_name, visit_time):
        """
        Add a visitor entry to the log.
        Parameters:
        visitor_name (str): The name of the visitor.
        visit_time (str): The time of the visit.
        """
        entry = {"name": visitor_name, "time": visit_time}
        self.visitor_entries.append(entry)
        print(f"Visitor added: {visitor_name} at {visit_time}")
    def view_visitors(self):
        """
        View all visitor entries in the log.
        Returns:
        list: A list of all visitor entries or a message if there are no
entries.
        """
        if self.visitor_entries:
            return self.visitor_entries
        else:
            return "No visitors have entered the hostel."
# Example usage
visitor_log = VisitorLog()
visitor_log.add_visitor("John Doe", "2024-06-01 10:00 AM")
visitor_log.add_visitor("Jane Smith", "2024-06-01 11:00 AM")
print(visitor_log.view_visitors())
```

Room 101 allotted to Alice.

Room 102 allotted to Bob.

Alice

Room 103 is vacant.

---15.1 done---

Medical emergency added: Student A has a severe headache.

Medical emergency added: Student B has a broken arm.

Medical emergency handled: Student A has a severe headache.

Medical emergency handled: Student B has a broken arm.

No medical emergencies to handle.

---15.2 done---

Student Alice added successfully.

Student Bob added successfully.

{'name': 'Alice', 'age': 20, 'course': 'Computer Science'}

Student not found.

---15.3 done---

['Block B', 'Block C']

['Block A', 'Block C']

['Block A', 'Block B']

Hostel block not found.

---15.4 done---

Visitor added: John Doe at 2024-06-01 10:00 AM

Visitor added: Jane Smith at 2024-06-01 11:00 AM

[{'name': 'John Doe', 'time': '2024-06-01 10:00 AM'}, {'name': 'Jane Smith', 'time': '2024-06-01 11:00 AM'}]

---15.5 done---

Feature	Data Structure	Justification
Room Allotment	Hash Table (Dictionary)	Room numbers act as keys mapping to student names, enabling O(1) allotment and lookup. Dictionary avoids duplicate assignments and supports instant availability checks.

Feature	Data Structure	Justification
Medical Emergency Handling	Queue (List)	Emergencies must be addressed in the order they are reported — FIFO. A list simulating a queue ensures the earliest reported case is handled first.
Student Record Retrieval	Hash Table (Dictionary)	Student IDs are unique keys for retrieving student records instantly. Dictionary provides O(1) average-case lookup, essential for fast record access in large databases.
Hostel Block Connectivity	Graph (Adjacency List)	Hostel blocks share facilities and are connected — modelled as graph nodes and edges. An adjacency list efficiently represents these connections and supports connectivity queries.
Visitor Entry Log	List (Dynamic Array)	Visitor entries are recorded in order of arrival and viewed chronologically. A list preserves insertion order and allows O(1) append and full iteration for log display.

Task #16: Smart Electricity Distribution System
Scenario:
An electricity board plans to develop a system that includes:
1. Complaint Handling
2. Fault Priority Handling
3. Consumer Record Lookup
4. Power Grid Connectivity
5. Meter Reading History Tracking
Student Task
• For each feature, select the most appropriate data structure from the list below:
o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque
• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:
• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.

16.1

```
"""
Smart Electricity Distribution System

Feature: Complaint Handling

Consumers submit complaints that must be processed in order.

Tasks:
- Choose an appropriate data structure for handling complaints.
- Implement a Python simulation for registering and processing complaints.
- Include comments and docstrings.
"""
class ComplaintHandler:
    """A class to manage consumer complaints in an electricity distribution system."""
    def __init__(self):
        """Initialize the complaint handler using a list to simulate a queue."""
```

```
        self.complaint_queue = []
    def register_complaint(self, complaint_description):
        """
        Register a new complaint in the queue.
        Parameters:
        complaint_description (str): The description of the consumer
complaint.
        """
        self.complaint_queue.append(complaint_description)
        print(f"Complaint registered: {complaint_description}")
    def process_complaint(self):
        """
        Process the next complaint in the queue.
        Returns:
        str: The description of the processed complaint or a message if there
are no complaints to process.
        """
        if self.complaint_queue:
            processed_complaint = self.complaint_queue.pop(0)
            print(f"Complaint processed: {processed_complaint}")
            return processed_complaint
        else:
            print("No complaints to process.")
            return None
# Example usage
complaint_handler = ComplaintHandler()
complaint_handler.register_complaint("Power outage in Sector 5.")
complaint_handler.register_complaint("Frequent voltage fluctuations in Sector
3.")
complaint_handler.process_complaint()
complaint_handler.process_complaint()
complaint_handler.process_complaint()
```

16.2

```
"""
Smart Electricity Distribution System

Feature: Fault Priority Handling

Critical faults must be repaired before normal issues.

Tasks:
- Choose a suitable data structure for managing fault priorities.
- Implement a Python simulation for handling faults.
- Include comments and docstrings.
"""
class FaultHandler:
```

```
    """A class to manage faults in an electricity distribution system."""
    def __init__(self):
        """Initialize the fault handler using a list to simulate a priority
queue."""
        self.fault_queue = []
    def add_fault(self, fault_description, priority):
        """
        Add a new fault to the queue with a specified priority.
        Parameters:
        fault_description (str): The description of the fault.
        priority (int): The priority level of the fault (lower number
indicates higher priority).
        """
        self.fault_queue.append((priority, fault_description))
        self.fault_queue.sort(key=lambda x: x[0])  # Sort by priority
        print(f"Fault added: {fault_description} with priority {priority}")
    def handle_fault(self):
        """
        Handle the next fault in the queue based on priority.
        Returns:
        str: The description of the handled fault or a message if there are no
faults to handle.
        """
        if self.fault_queue:
            handled_fault = self.fault_queue.pop(0)
            print(f"Fault handled: {handled_fault[1]} with priority
{handled_fault[0]}")
            return handled_fault[1]
        else:
            print("No faults to handle.")
            return None
# Example usage
fault_handler = FaultHandler()
fault_handler.add_fault("Power outage in Sector 5.", priority=1)
fault_handler.add_fault("Frequent voltage fluctuations in Sector 3.",
priority=2)
fault_handler.handle_fault()
fault_handler.handle_fault()
fault_handler.handle_fault()
```

16.3

```
"""
Smart Electricity Distribution System

Feature: Consumer Record Lookup

The system must quickly find consumer records using a consumer ID.
```

```
Tasks:
- Choose a data structure suitable for fast record lookup.
- Implement a Python program to store and retrieve consumer records.
- Include comments and docstrings.
"""
class ConsumerRecord:
    """A class to manage consumer records in an electricity distribution
system."""
    def __init__(self):
        """Initialize the consumer record management system."""
        self.consumer_db = {}
    def add_consumer(self, consumer_id, name, address, contact):
        """
        Add a consumer record to the database.
        Parameters:
        consumer_id (str): The unique identifier for the consumer.
        name (str): The name of the consumer.
        address (str): The address of the consumer.
        contact (str): The contact information of the consumer.
        """
        self.consumer_db[consumer_id] = {
            "name": name,
            "address": address,
            "contact": contact
        }
        print(f"Consumer {name} added successfully.")
    def get_consumer_record(self, consumer_id):
        """
        Retrieve a consumer record by Consumer ID.
        Parameters:
        consumer_id (str): The unique identifier for the consumer to look up.
        Returns:
        dict: A dictionary containing the consumer's name, address, and
contact,
              or a message if the consumer is not found.
        """
        if consumer_id in self.consumer_db:
            return self.consumer_db[consumer_id]
        else:
            return "Consumer not found."
# Example usage
consumer_record = ConsumerRecord()
consumer_record.add_consumer("C001", "Alice", "123 Main St", "555-1234")
consumer_record.add_consumer("C002", "Bob", "456 Elm St", "555-5678")
print(consumer_record.get_consumer_record("C001"))
print(consumer_record.get_consumer_record("C003"))
```

16.4

```
"""
Smart Electricity Distribution System

Feature: Power Grid Connectivity

Power stations and substations are connected through a network.

Tasks:
- Choose a data structure suitable for representing the power grid network.
- Implement a Python representation of the grid connectivity.
- Include comments and docstrings.
"""
class PowerGrid:
    """A class to represent the power grid network of stations and
substations."""
    def __init__(self):
        """Initialize the power grid using an adjacency list."""
        self.grid = {}
    def add_connection(self, station1, station2):
        """
        Add a bidirectional connection between two stations.
        Parameters:
        station1 (str): The name of the first station.
        station2 (str): The name of the second station.
        """
        if station1 not in self.grid:
            self.grid[station1] = []
        if station2 not in self.grid:
            self.grid[station2] = []
        self.grid[station1].append(station2)
        self.grid[station2].append(station1)
    def get_connections(self, station):
        """
        Retrieve the list of connected stations for a given station.
        Parameters:
        station (str): The name of the station to look up.
        Returns:
        list: A list of connected stations or a message if the station is not
found.
        """
        if station in self.grid:
            return self.grid[station]
        else:
            return "Station not found."
# Example usage
power_grid = PowerGrid()
power_grid.add_connection("Station A", "Substation 1")
```

```
power_grid.add_connection("Station A", "Substation 2")
power_grid.add_connection("Substation 1", "Substation 3")
print(power_grid.get_connections("Station A"))
print(power_grid.get_connections("Substation 1"))
print(power_grid.get_connections("Substation 2"))
print(power_grid.get_connections("Substation 3"))
```

16.5

```
"""
Smart Electricity Distribution System

Feature: Meter Reading History Tracking

The system must store and track historical meter readings for consumers.

Tasks:
- Choose a data structure suitable for storing meter history.
- Implement a Python program to add and view meter readings.
- Include comments and docstrings.
"""
class MeterReadingHistory:
    """A class to store and track historical meter readings for consumers."""
    def __init__(self):
        """Initialize the meter reading history using a dictionary."""
        self.readings = {}
    def add_reading(self, consumer_id, reading):
        """
        Add a new meter reading for a consumer.
        Parameters:
        consumer_id (str): The unique identifier for the consumer.
        reading (float): The meter reading value to be added.
        """
        if consumer_id not in self.readings:
            self.readings[consumer_id] = []
        self.readings[consumer_id].append(reading)
        print(f"Meter reading added for consumer {consumer_id}: {reading}")
    def view_readings(self, consumer_id):
        """
        View the historical meter readings for a given consumer.
        Parameters:
        consumer_id (str): The unique identifier for the consumer.
        Returns:
        list: A list of meter readings for the specified consumer or a message
if the consumer is not found.
        """
        if consumer_id in self.readings:
            return self.readings[consumer_id]
```

```
        else:
            return "Consumer not found."
# Example usage
meter_history = MeterReadingHistory()
meter_history.add_reading("C001", 150.5)
meter_history.add_reading("C001", 155.0)
meter_history.add_reading("C002", 200.0)
print(meter_history.view_readings("C001"))
print(meter_history.view_readings("C002"))
print(meter_history.view_readings("C003"))
```

Complaint registered: Power outage in Sector 5.

Complaint registered: Frequent voltage fluctuations in Sector 3.

Complaint processed: Power outage in Sector 5.

Complaint processed: Frequent voltage fluctuations in Sector 3.

No complaints to process.

---16.1 done---

Fault added: Power outage in Sector 5. with priority 1

Fault added: Frequent voltage fluctuations in Sector 3. with priority 2

Fault handled: Power outage in Sector 5. with priority 1

Fault handled: Frequent voltage fluctuations in Sector 3. with priority 2

No faults to handle.

---16.2 done---

Consumer Alice added successfully.

Consumer Bob added successfully.

{'name': 'Alice', 'address': '123 Main St', 'contact': '555-1234'}

Consumer not found.

---16.3 done---

['Substation 1', 'Substation 2']

['Station A', 'Substation 3']

['Station A']

['Substation 1']

---16.4 done---

Meter reading added for consumer C001: 150.5

Meter reading added for consumer C001: 155.0

Meter reading added for consumer C002: 200.0

[150.5, 155.0]

[200.0]

Consumer not found.

---16.5 done---

Feature	Data Structure	Justification
Complaint Handling	Queue (List)	Consumer complaints must be processed in the order they are received — FIFO. A list simulating a queue ensures fair processing and prevents complaint starvation.
Fault Priority Handling	Priority Queue (Sorted List)	Critical faults must be repaired before minor ones regardless of arrival order. A sorted list ordered by priority level ensures the most critical fault is always handled first.
Consumer Record Lookup	Hash Table (Dictionary)	Consumer IDs are unique keys for retrieving consumer details instantly. Dictionary provides O(1) average-case lookup, making it ideal for large-scale consumer databases.
Power Grid Connectivity	Graph (Adjacency List)	Power stations and substations form a network of interconnected nodes. An adjacency list efficiently models this sparse network and supports failure analysis and traversal.
Meter Reading History Tracking	Hash Table + List (Dictionary of Lists)	Each consumer has a sequence of historical readings. A dictionary keyed by consumer ID maps to a list of readings, enabling O(1) consumer lookup and ordered history access.

Task #17: Smart Logistics & Warehouse Management
Scenario:
A logistics company wants to implement:
1. Shipment Processing
2. Urgent Shipment Priority
3. Product Lookup by Code
4. Warehouse Network Connectivity
5. Inventory Add/Remove Operations
Student Task
• For each feature, select the most appropriate data structure from the list below:
o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque
• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.
Expected Output:
• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.

17.1

```
"""
Smart Logistics and Warehouse Management System

Feature: Shipment Processing

Shipments must be processed in the order they arrive.

Tasks:
- Choose an appropriate data structure for shipment processing.
- Implement a Python simulation to add and process shipments.
- Include comments and docstrings.
"""
class ShipmentProcessor:
    """A class to manage shipment processing in a logistics and warehouse management system."""
    def __init__(self):
        """Initialize the shipment processor using a list to simulate a queue."""
```

```
        self.shipment_queue = []
    def add_shipment(self, shipment_description):
        """
        Add a new shipment to the processing queue.
        Parameters:
        shipment_description (str): The description of the shipment to be
processed.
        """
        self.shipment_queue.append(shipment_description)
        print(f"Shipment added: {shipment_description}")
    def process_shipment(self):
        """
        Process the next shipment in the queue.
        Returns:
        str: The description of the processed shipment or a message if there
are no shipments to process.
        """
        if self.shipment_queue:
            processed_shipment = self.shipment_queue.pop(0)
            print(f"Shipment processed: {processed_shipment}")
            return processed_shipment
        else:
            print("No shipments to process.")
            return None
# Example usage
shipment_processor = ShipmentProcessor()
shipment_processor.add_shipment("Shipment A: 100 units of product X.")
shipment_processor.add_shipment("Shipment B: 50 units of product Y.")
shipment_processor.process_shipment()
shipment_processor.process_shipment()
shipment_processor.process_shipment()
```

17.2

```
"""
Smart Logistics and Warehouse Management System

Feature: Urgent Shipment Priority

Urgent shipments must be processed before normal shipments.

Tasks:
- Choose a suitable data structure for managing shipment priority.
- Implement a Python simulation for handling urgent shipments.
- Include comments and docstrings.
"""
class ShipmentHandler:
```

```
    """A class to manage shipments in a logistics and warehouse management
system."""
    def __init__(self):
        """Initialize the shipment handler using a list to simulate a priority
queue."""
        self.shipment_queue = []
    def add_shipment(self, shipment_description, priority):
        """
        Add a new shipment to the queue with a specified priority.
        Parameters:
        shipment_description (str): The description of the shipment.
        priority (int): The priority level of the shipment (lower number
indicates higher priority).
        """
        self.shipment_queue.append((priority, shipment_description))
        self.shipment_queue.sort(key=lambda x: x[0])  # Sort by priority
        print(f"Shipment added: {shipment_description} with priority
{priority}")
    def process_shipment(self):
        """
        Process the next shipment in the queue based on priority.
        Returns:
        str: The description of the processed shipment or a message if there
are no shipments to process.
        """
        if self.shipment_queue:
            processed_shipment = self.shipment_queue.pop(0)
            print(f"Shipment processed: {processed_shipment[1]} with priority
{processed_shipment[0]}")
            return processed_shipment[1]
        else:
            print("No shipments to process.")
            return None
# Example usage
shipment_handler = ShipmentHandler()
shipment_handler.add_shipment("Urgent delivery to Client A.", priority=1)
shipment_handler.add_shipment("Regular delivery to Client B.", priority=2)
shipment_handler.process_shipment()
shipment_handler.process_shipment()
shipment_handler.process_shipment()
```

17.3

```
"""
Smart Logistics and Warehouse Management System

Feature: Product Lookup by Code
```

```
Warehouse staff must quickly find product information using a product code.

Tasks:
- Choose a data structure suitable for fast product lookup.
- Implement a Python program to store and retrieve product details.
- Include comments and docstrings.
"""
class ProductCatalog:
    """A class to manage the product catalog for a logistics and warehouse
management system."""
    def __init__(self):
        """Initialize the product catalog using a dictionary for fast
lookup."""
        self.product_db = {}
    def add_product(self, product_code, name, quantity, price):
        """
        Add a product to the catalog.
        Parameters:
        product_code (str): The unique code for the product.
        name (str): The name of the product.
        quantity (int): The quantity of the product in stock.
        price (float): The price of the product.
        """
        self.product_db[product_code] = {
            "name": name,
            "quantity": quantity,
            "price": price
        }
        print(f"Product {name} added successfully.")
    def get_product_info(self, product_code):
        """
        Retrieve product information by Product Code.
        Parameters:
        product_code (str): The unique code for the product to look up.
        Returns:
        dict: A dictionary containing the product's name, quantity, and price,
              or a message if the product is not found.
        """
        if product_code in self.product_db:
            return self.product_db[product_code]
        else:
            return "Product not found."
# Example usage
product_catalog = ProductCatalog()
product_catalog.add_product("P001", "Widget A", 100, 2.99)
product_catalog.add_product("P002", "Widget B", 50, 4.99)
print(product_catalog.get_product_info("P001"))
print(product_catalog.get_product_info("P002"))
```

```
print(product_catalog.get_product_info("P003"))
```

17.4

```
"""
Smart Logistics and Warehouse Management System

Feature: Warehouse Network Connectivity

Multiple warehouses are connected through transport routes.

Tasks:
- Choose a suitable data structure to represent warehouse connections.
- Implement a Python representation of the warehouse network.
- Include comments and docstrings.
"""
class WarehouseNetwork:
    """A class to represent the network of warehouses and their
connections."""
    def __init__(self):
        """Initialize the warehouse network using an adjacency list."""
        self.network = {}
    def add_connection(self, warehouse1, warehouse2):
        """
        Add a bidirectional connection between two warehouses.
        Parameters:
        warehouse1 (str): The name of the first warehouse.
        warehouse2 (str): The name of the second warehouse.
        """
        if warehouse1 not in self.network:
            self.network[warehouse1] = []
        if warehouse2 not in self.network:
            self.network[warehouse2] = []
        self.network[warehouse1].append(warehouse2)
        self.network[warehouse2].append(warehouse1)
    def get_connections(self, warehouse):
        """
        Retrieve the list of connected warehouses for a given warehouse.
        Parameters:
        warehouse (str): The name of the warehouse to look up.
        Returns:
        list: A list of connected warehouses or a message if the warehouse is
not found.
        """
        if warehouse in self.network:
            return self.network[warehouse]
        else:
            return "Warehouse not found."
```

```
# Example usage
warehouse_network = WarehouseNetwork()
warehouse_network.add_connection("Warehouse A", "Warehouse B")
warehouse_network.add_connection("Warehouse A", "Warehouse C")
warehouse_network.add_connection("Warehouse B", "Warehouse D")
print(warehouse_network.get_connections("Warehouse A"))
print(warehouse_network.get_connections("Warehouse B"))
print(warehouse_network.get_connections("Warehouse C"))
```

17.5

```
"""
Smart Logistics and Warehouse Management System

Feature: Inventory Add and Remove Operations

Products are added to and removed from warehouse inventory.

Tasks:
- Choose a suitable data structure for managing inventory operations.
- Implement a Python program to add and remove products.
- Include comments and docstrings.
"""
class InventoryManager:
    """A class to manage inventory operations for a logistics and warehouse
management system."""
    def __init__(self):
        """Initialize the inventory manager using a dictionary to store
product quantities."""
        self.inventory = {}
    def add_product(self, product_code, quantity):
        """
        Add a specified quantity of a product to the inventory.
        Parameters:
        product_code (str): The unique code for the product.
        quantity (int): The quantity of the product to add.
        """
        if product_code in self.inventory:
            self.inventory[product_code] += quantity
        else:
            self.inventory[product_code] = quantity
        print(f"Added {quantity} units of product {product_code}.")
    def remove_product(self, product_code, quantity):
        """
        Remove a specified quantity of a product from the inventory.
        Parameters:
        product_code (str): The unique code for the product.
        quantity (int): The quantity of the product to remove.
```

```
        Returns:
        str: A message indicating the result of the operation.
        """
        if product_code in self.inventory:
            if self.inventory[product_code] >= quantity:
                self.inventory[product_code] -= quantity
                print(f"Removed {quantity} units of product {product_code}.")
                return f"{quantity} units of product {product_code} removed."
            else:
                print(f"Not enough stock to remove {quantity} units of product
{product_code}.")
                return f"Not enough stock to remove {quantity} units of
product {product_code}."
        else:
            print(f"Product {product_code} not found in inventory.")
            return f"Product {product_code} not found in inventory."
# Example usage
inventory_manager = InventoryManager()
inventory_manager.add_product("P001", 100)
inventory_manager.add_product("P002", 50)
inventory_manager.remove_product("P001", 20)
```

Shipment added: Shipment A: 100 units of product X.

Shipment added: Shipment B: 50 units of product Y.

Shipment processed: Shipment A: 100 units of product X.

Shipment processed: Shipment B: 50 units of product Y.

No shipments to process.

---17.1 done---

Shipment added: Urgent delivery to Client A. with priority 1

Shipment added: Regular delivery to Client B. with priority 2

Shipment processed: Urgent delivery to Client A. with priority 1

Shipment processed: Regular delivery to Client B. with priority 2

No shipments to process.

---17.2 done---

Product Widget A added successfully.

Product Widget B added successfully.

{'name': 'Widget A', 'quantity': 100, 'price': 2.99}

{'name': 'Widget B', 'quantity': 50, 'price': 4.99}

Product not found.

---17.3 done---

['Warehouse B', 'Warehouse C']

['Warehouse A', 'Warehouse D']

['Warehouse A']

---17.4 done---

Added 100 units of product P001.

Added 50 units of product P002.

Removed 20 units of product P001.

---17.5 done---

Feature	Data Structure	Justification
Shipment Processing	Queue (List)	Shipments must be dispatched in the order they arrive — FIFO. A list simulating a queue ensures first-arrived shipments are processed first, maintaining fairness.
Urgent Shipment Priority	Priority Queue (Sorted List)	Urgent shipments must be dispatched before regular ones regardless of arrival time. Sorting by priority level guarantees the highest-priority shipment is always processed next.
Product Lookup by Code	Hash Table (Dictionary)	Product codes serve as unique keys for retrieving product details instantly. Dictionary provides O(1) average-case lookup, essential for fast warehouse inventory queries.
Warehouse Network Connectivity	Graph (Adjacency List)	Warehouses are connected by transport routes — nodes and edges in a graph. An adjacency list efficiently represents this sparse network and supports route planning queries.
Inventory Add/Remove Operations	Hash Table (Dictionary)	Product codes map to quantities, enabling O(1) update for add/remove operations. Dictionary structure avoids scanning the entire inventory for every stock change.

Test Cases Design
Task 18 (Collatz Sequence Generator – Test Case Design)
• Function: Generate Collatz sequence until reaching 1.
• Test Cases to Design:
• Normal: 6 → [6,3,10,5,16,8,4,2,1]
• Edge: 1 → [1]
• Negative: -5
• Large: 27 (well-known long sequence)
• Requirement: Validate correctness with pytest.
Explanation:
We need to write a function that:
• Takes an integer n as input.
• Generates the Collatz sequence (also called the 3n+1 sequence).
• The rules are:
o If n is even → next = n / 2.
o If n is odd → next = 3n + 1.
• Repeat until we reach 1.
• Return the full sequence as a list.
Example
Input: 6
Steps:
• 6 (even → 6/2 = 3)
• 3 (odd → 3*3+1 = 10)
• 10 (even → 10/2 = 5)
• 5 (odd → 3*5+1 = 16)
• 16 (even → 16/2 = 8)
• 8 (even → 8/2 = 4)
• 4 (even → 4/2 = 2)
• 2 (even → 2/2 = 1)
Output:
[6, 3, 10, 5, 16, 8, 4, 2, 1]

```
"""
Collatz Sequence Generator with Test Cases

Write a Python function collatz_sequence(n) that generates the Collatz
(3n + 1) sequence until the number reaches 1.

Rules:
- If n is even → next number = n / 2
- If n is odd → next number = 3*n + 1
- Continue until the sequence reaches 1
- Return the full sequence as a list

Example:
Input: 6
Output: [6, 3, 10, 5, 16, 8, 4, 2, 1]
```

```
Requirements:
- Implement the function collatz_sequence(n)
- Handle invalid inputs such as negative numbers
- Write pytest test cases for:
    Normal case: 6 → [6,3,10,5,16,8,4,2,1]
    Edge case: 1 → [1]
    Negative case: -5 → should raise an error
    Large case: 27 → long sequence
- Use pytest to validate correctness.
"""
def collatz_sequence(n):
    """Generate the Collatz sequence for a given number n.
    Parameters:
    n (int): The starting number for the Collatz sequence. Must be a positive
integer.
    Returns:
    list: A list containing the Collatz sequence starting from n and ending at
1.
    Raises:
    ValueError: If n is not a positive integer.
    """
    if n <= 0:
        raise ValueError("Input must be a positive integer.")

    sequence = []
    while n != 1:
        sequence.append(n)
        if n % 2 == 0:
            n = n // 2
        else:
            n = 3 * n + 1
    sequence.append(1)  # Append the final element of the sequence
    return sequence
# Example usage
if __name__ == "__main__":
    try:
        print(collatz_sequence(6))  # Normal case
        print(collatz_sequence(1))  # Edge case
        print(collatz_sequence(-5)) # Negative case (should raise an error)
    except ValueError as e:
        print(e)
    print(collatz_sequence(27))  # Large case
```

Task 19 (Lucas Number Sequence – Test Case Design)
• Function: Generate Lucas sequence up to n terms.
(Starts with 2,1, then Fn = Fn-1 + Fn-2)
• Test Cases to Design:
• Normal: 5 → [2, 1, 3, 4, 7]
• Edge: 1 → [2]
• Negative: -5 → Error
• Large: 10 (last element = 76).
• Requirement: Validate correctness with pytest.

```
"""
Lucas Number Sequence Generator with Test Cases

Write a Python function lucas_sequence(n) that generates the first n
terms of the Lucas sequence.

Lucas sequence rules:
- First two numbers are 2 and 1
- Each next number = previous two numbers added
  Fn = Fn-1 + Fn-2

Example:
Input: 5
Output: [2, 1, 3, 4, 7]

Requirements:
- Implement lucas_sequence(n) that returns a list
- Handle invalid input such as negative numbers
- Write pytest test cases for:
    Normal case: 5 → [2,1,3,4,7]
    Edge case: 1 → [2]
    Negative case: -5 → should raise an error
    Large case: 10 → last element should be 76
- Use pytest to validate correctness.
"""
def lucas_sequence(n):
    """Generate the first n terms of the Lucas sequence.
    Parameters:
    n (int): The number of terms to generate. Must be a non-negative integer.
    Returns:
    list: A list containing the first n terms of the Lucas sequence.
    Raises:
    ValueError: If n is negative.
    """
    if n < 0:
        raise ValueError("Input must be a non-negative integer.")

    if n == 0:
        return []
```

```
    elif n == 1:
        return [2]
    elif n == 2:
        return [2, 1]

    sequence = [2, 1]
    for i in range(2, n):
        next_term = sequence[i-1] + sequence[i-2]
        sequence.append(next_term)

    return sequence
# Example usage
if __name__ == "__main__":
    try:
        print(lucas_sequence(5))  # Normal case
        print(lucas_sequence(1))  # Edge case
        print(lucas_sequence(-5)) # Negative case (should raise an error)
    except ValueError as e:
        print(e)
    print(lucas_sequence(10))  # Large case
```

Task 20 (Vowel & Consonant Counter – Test Case Design)
• Function: Count vowels and consonants in string.
• Test Cases to Design:
• Normal: "hello" → (2,3)
• Edge: "" → (0,0)
• Only vowels: "aeiou" → (5,0)
Large: Long text
• Requirement: Validate correctness with pytest.

```
"""
Vowel and Consonant Counter with Test Cases

Write a Python function count_vowels_consonants(text) that counts
the number of vowels and consonants in a given string.

Rules:
- Vowels: a, e, i, o, u
- All other alphabet characters are consonants
- Ignore spaces and special characters if needed

Example:
Input: "hello"
Output: (2,3)

Requirements:
- Implement the function count_vowels_consonants(text)
- Return a tuple (vowel_count, consonant_count)
- Write pytest test cases for:
    Normal case: "hello" → (2,3)
    Edge case: "" → (0,0)
    Only vowels: "aeiou" → (5,0)
    Large case: long paragraph text
- Use pytest to validate correctness.
"""
def count_vowels_consonants(text):
    """Count the number of vowels and consonants in a given string.
    Parameters:
    text (str): The input string to analyze.
    Returns:
    tuple: A tuple containing the count of vowels and consonants (vowel_count,
consonant_count).
    """
    vowels = 'aeiouAEIOU'
    vowel_count = 0
    consonant_count = 0

    for char in text:
        if char.isalpha():  # Check if the character is an alphabet
            if char in vowels:
```

```
                vowel_count += 1
            else:
                consonant_count += 1

    return (vowel_count, consonant_count)
# Example usage
if __name__ == "__main__":
    print(count_vowels_consonants("hello"))  # Normal case
    print(count_vowels_consonants(""))       # Edge case
    print(count_vowels_consonants("aeiou"))  # Only vowels case
    # Large case can be tested with a long paragraph of text
    print(count_vowels_consonants("This is a large paragraph with many
words."))
```

=== pytest 18/19/20 ===

============================= test session starts ==============================

platform win32 -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\Surya Teja\AppData\Local\Programs\Python\Python313\python.exe

cachedir: .pytest_cache

rootdir: Z:\AIAC\Assignment 13

plugins: anyio-4.11.0

collected 12 items

test_18.py::test_normal_case PASSED [8%]

test_18.py::test_edge_case_1 PASSED [16%]

test_18.py::test_negative_raises PASSED [25%]

test_18.py::test_large_case PASSED [33%]

test_19.py::test_normal_case PASSED [41%]

test_19.py::test_edge_case_1 PASSED [50%]

test_19.py::test_negative_raises PASSED [58%]

test_19.py::test_large_case PASSED [66%]

test_20.py::test_normal_case PASSED [75%]

test_20.py::test_edge_case_empty PASSED [83%]

test_20.py::test_only_vowels PASSED [91%]

test_20.py::test_large_case PASSED [100%]

============================== 12 passed in 0.09s ==============================